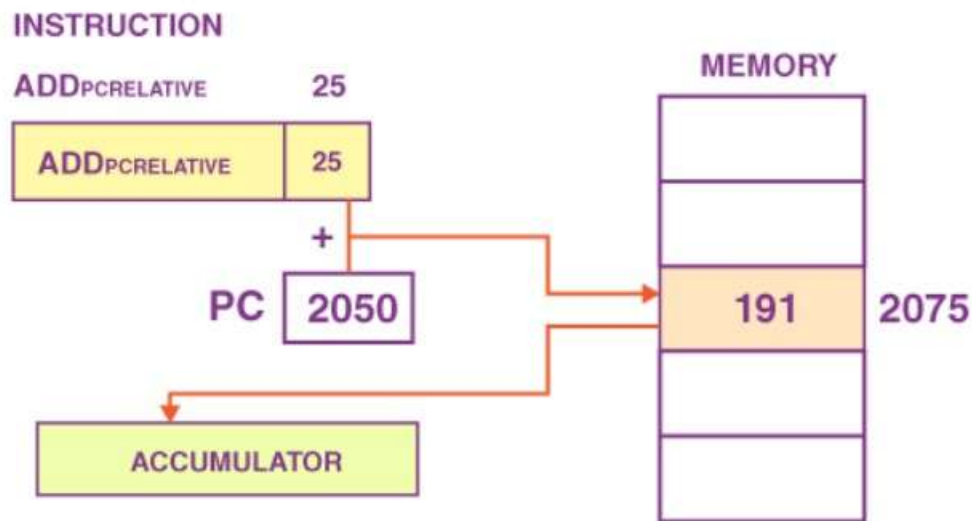


Computer System Architecture
COMP201Th
Unit: 3
Programming the Basic Computer
Lecture: 2 (Part-II)
Addressing Modes

Relative Addressing Mode: In this mode, the Effective Address (EA) of the operand is calculated by adding the content of the CPU register and the address part of the instruction word. The effective address is calculated by adding displacement (immediate value given in the instruction) and the register value.

So $EA = \text{Displacement} + \text{CPU Register}$



Symbolic representation for the relative address mode:

$X(PC)$

then effective address will be: $EA = X + PC$

Since the operand addresses in this mode are found relative to a program counter, it is known as the relative addressing mode.

Indexed Addressing Mode: In the indexed addressing mode, the content of a given index register gets added to an instruction's address part so as to obtain the effective address.

Here, the index refers to a special CPU register that consists of an index value.

The index addressing mode is helpful whenever the instructions in a program access an array or large ranges of memory addresses. The effective address in

such a mode, is generated when we add a constant to the content of the register. The register content does not ever change.

Symbolic representation for indexed addressing mode:

$X(R)$

The effective address will be: $EA = X + (R)$

e.g.

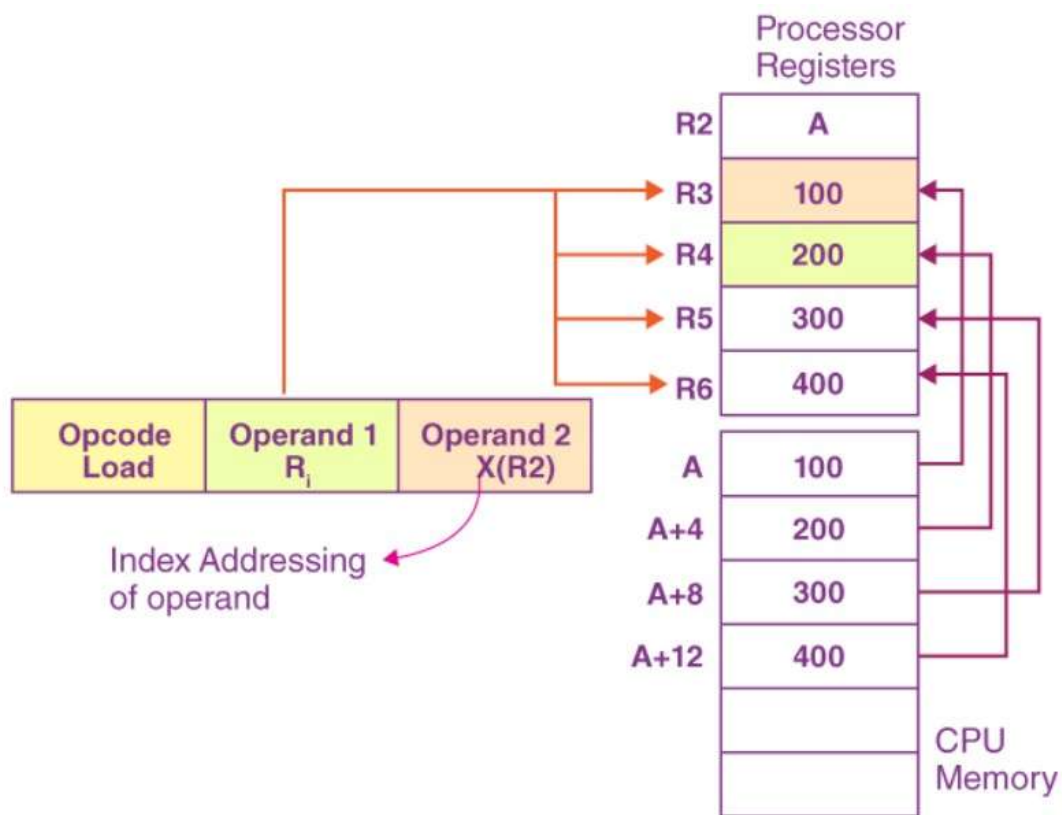
Load R2, A

Load R3, (R2)

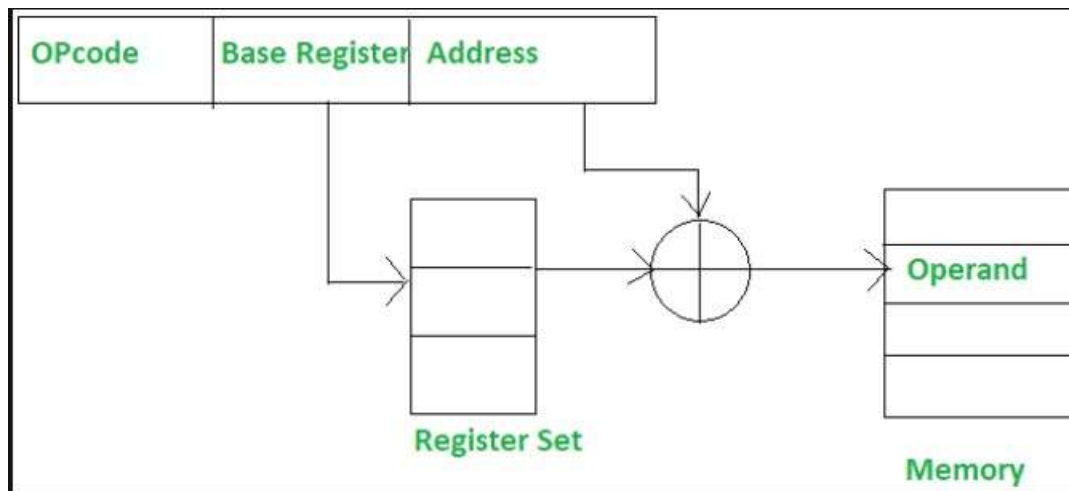
Load R4, 4(R2)

Load R5, 8(R2)

Load R6, 12(R2)



Base Register Addressing Mode: In this addressing mode, the effective address is obtained by adding base register value to address field value.



Auto-increment Addressing Mode: In this mode, the content present in the register is initially incremented, and then the content that is incremented in the register is used in the form of an effective address.

Once the content present in the register in the auto-increment addressing mode is accessed by its instruction, the content of the register is incremented to refer to the next operand.

Symbolically, we represent it as follows:

$(R)+$

The auto increment mode helps us in implementing the stack structure.

Auto-decrement Addressing Mode: It's the opposite of auto-increment addressing mode. In this, the content present in the register is initially decremented, and then the content that is decremented in the register is used in the form of an effective address.

Symbolically, we represent it as:

$-(R)$